

Java

Exception handling and access specifiers

Theory

1. Enums in Java

Definition: An enum is a special data type that represents a fixed set of constants. **Use case:** Improves readability and type safety compared to using raw integers or strings. **Key point:** Enums can be used in switch statements and can have methods/constructors.

Syntax:

```
enum LEVEL { HIGH, MID, LOW }
```

2. Exception Handling

Definition: An exception is an event that disrupts the normal flow of a program. **Use case:** Helps handle runtime errors gracefully without crashing the program. **Key point:**

- **Checked exceptions** → verified at compile time (e.g., IOException).
- **Unchecked exceptions** → occur at runtime (e.g., ArithmeticException).

Syntax:

```
try {  
    int result = 10/0;  
} catch(ArithmeticException e) {  
    System.out.println("Error: " + e.getMessage());  
} finally {  
    System.out.println("Cleanup code");  
}
```

3. Access Specifiers

Definition: Access specifiers define the visibility of classes, methods, and variables.

Types:

- **private** → accessible only within the class.
- **protected** → accessible within the package and subclasses.
- **public** → accessible everywhere.
- **default (no modifier)** → accessible within the package only.

Syntax:

```
public class Example {  
    private int data;  
    protected String name;  
    public void greet(){ ... }  
    void display(){ ... } // default  
}
```

Coding Round Questions

- Enum usage → Create an enum for traffic lights and print its values.
- Throw vs Throws → Demonstrate difference between throw and throws.
- Multiple exceptions → Write a program to handle ArithmeticException, ArrayIndexOutOfBoundsException, and NumberFormatException.
- Access specifiers → Explain how private, protected, public, and default work with examples.
- Custom exception → Create a custom exception InvalidAgeException and use it in a program.

Machine Coding Round Challenge

Problem: Bank Account System with Access Control

Requirements:

- Create a BankAccount class with:

- private balance field.
 - public deposit() and withdraw() methods.
 - protected accountHolderName.
- Throw a custom exception InsufficientFundsException if withdrawal exceeds balance.
- Use an enum ACCOUNT_TYPE { SAVINGS, CURRENT }.
- Demonstrate exception handling and access specifiers in action.

```
enum LEVEL{
    HIGH,
    MID,
    LOW
}

public class ExceptionHandling {

    static void display(int age){
        if(age < 18){
            throw new ArithmeticException("Access denied");
        }
    }

    public static void main(String[] args) {

        int data = 10;
        LEVEL l = LEVEL.HIGH;
        System.out.println(l);

        // ExceptionHandling obj = new ExceptionHandling();
        // obj.display(10);

        display(20);
        System.out.println("Exception handling");

        try {

            String num = "abc";
            int resultNum = Integer.parseInt(num);
            int arr[] = {1,2,3};
            System.out.println(arr[5]);
        }
    }
}
```

```

        System.out.println(10/0);
    }catch(ArithmeticException e){
        System.out.println(e.getMessage());
        System.out.println("dlkafjsdlkfajflk");
    }catch(ArrayIndexOutOfBoundsException e){
        System.out.println(e.getMessage());
    }catch(Exception e){
        System.out.println(e.getMessage());
    }

    // finally{
    //     System.out.println("Exception is over");
    // }
    System.out.println("End of the program ");

}

}

```

User.java

```

package user;

public class Accessspecificier {

    private int data = 10;

    protected String name = "FIIT";

    public void greet(){
        System.out.println("Welcome to FIIT");
    }

    void display(){
        System.out.println("Display method from AccessSpecifier class");
    }

}

```

Admin.java

```

package admin;

import user.Accessspecificier;

class ProtectedExample extends Accessspecificier{

```

```
}
```

```
public class AdminSpecifer {
```

```
    public static void main(String[] args) {
```

```
        Accessspecificier obj = new Accessspecificier();
```

```
        // obj.display();
```

```
        // System.out.println(obj.name);
```

```
        obj.greet();
```

```
        // ProtectedExample obj1 = new ProtectedExample();
```

```
        // System.out.println(obj1.name);
```

```
    }
```

```
}
```